DEPARTMENT OF COMPUTER APPLICATIONS

[INSTITUTION / COLLEGE NAME PLACEHOLDER]

MEDIGUIDE CARE

A Doctor–Patient Consultation and Follow-up Management System

A Mini-Project Report submitted in partial fulfillment of the requirements
for the award of the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

Submitted by:
Kashyap Patil
Roll No / USN: [Roll Number Placeholder]
MCA Semester IV
Academic Year: 2025 – 2026

Under the Guidance of:
[Project Guide Name Placeholder]
Designation: Assistant Professor
Department of Computer Applications
[Institution Name Placeholder]

October 2026

CERTIFICATE

This is to certify that the mini-project report entitled **"MediGuide Care: Doctor–Patient Consultation and Follow-up Management System"** is a bonafide record of work carried out by **Kashyap Patil** (Roll No: [Roll Number Placeholder]) in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications (MCA)** by [University / Institution Name Placeholder] during the academic year 2025 – 2026.

The project work described in this report has been carried out under our supervision and guidance, and has not formed the basis for the award of any other degree, diploma, associate-ship, or similar title previously.

[Project Guide Name] Internal Project Guide Dept. of Computer Applications	**[Head of Department Name]** Head of Department Dept. of Computer Applications
Internal Examiner Date: _____	**External Examiner** Date: _____

DECLARATION

I hereby declare that the project work presented in this report entitled **"MediGuide Care: Doctor–Patient Consultation and Follow-up Management System"** is an authentic record of original work carried out by me under the guidance of **[Project Guide Name Placeholder]**, Department of Computer Applications, [Institution Name Placeholder]. I further declare that this report has not been submitted previously to any other university or institute for any degree or examination.

Date: _____

Place: _____

Kashyap Patil

MCA Final Year, Roll No: [Roll Number Placeholder]

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, **[Project Guide Name Placeholder]**, for continuous encouragement, valuable suggestions, and intellectual guidance throughout the design, implementation, and documentation of MediGuide Care.

I also express my heartfelt thanks to **[Head of Department Name Placeholder]**, Head of the Department of Computer Applications, and the faculty members of the institution for providing the required infrastructure, laboratory computing facilities, and constructive feedback.

Finally, I extend my appreciation to my family and fellow classmates for their moral support, collaboration, and insightful discussions during the development of this project.

ABSTRACT

Outpatient medical clinics frequently grapple with fragmented scheduling, double-booked consultation intervals, unstructured paper-based prescriptions, and compromised patient privacy resulting from uncontrolled disclosure of personal phone numbers. **MediGuide Care** is a modern, full-stack, multi-tier web application engineered to resolve these operational deficiencies by unifying doctor discovery, availability planning, conflict-free appointment booking, digital consultation records, and AI-assisted clinical history summarization into a secure, cohesive platform.

The system is built upon a decoupled architecture comprising a single-page application client implemented with React 19 and Vite, a robust enterprise backend API powered by Java 17 and Spring Boot 3.2.5, and a transactional relational database persistence layer utilizing PostgreSQL 16 mapped via Spring Data JPA and Hibernate ORM. Security is enforced through stateless JSON Web Token (JWT) authentication signed with HMAC-SHA256, salted BCrypt password hashing, and fine-grained method-level Role-Based Access Control (RBAC). To safeguard user identity against sequential database enumeration, public-facing operations strictly utilize collision-resistant alphanumeric identifiers (DOC-XXXXXX and PAT-XXXXXX). A multi-stage email verification architecture—implemented with cryptographic 32-byte Base64URL tokens hashed with SHA-256 and subject to a 60-second rate-limiting cooldown—enforces progressive access control where high-trust operations such as appointment booking and schedule publication require confirmed accounts.

To guarantee zero appointment double-booking under concurrent traffic, the booking engine coordinates transactions via pessimistic row-level locking on ascending user identifiers and executes transactional range queries. Attending physicians document clinical findings, diagnoses, and structured digital prescriptions via a slide-over consultation drawer. An integrated OpenAI-compatible clinical summarizer extracts chronological patient records, strips sensitive identifiers, applies strict medical safety prompting rules, and yields non-binding draft summaries with doctor-in-the-loop verification disclaimers. The platform's stability and functional correctness are substantiated by an automated backend test suite comprising **298 passing automated tests (100% pass rate)** and static linting with zero warnings.

Keywords: *Healthcare Management, Spring Boot, React, PostgreSQL, JWT Authentication, Pessimistic Locking, Digital Prescriptions, AI Clinical Summarization, Email Verification.*

TABLE OF CONTENTS

Title	Page No.
Certificate	ii
Declaration	iii
Acknowledgement	iii
Abstract	iv
List of Figures	viii
List of Tables	viii
Chapter 1: Introduction	**1**
1.1 Project Overview and Domain Context	1
1.2 Purpose and Objectives	1
1.3 Target Users and Operating Context	2
1.4 Organization of the Report	2
Chapter 2: Problem Statement, Objectives, Scope, and Motivation	**3**
2.1 Problem Statement and Real-World Clinic Challenges	3
2.2 Project Objectives	3
2.3 Project Scope	4
2.4 Academic and Practical Motivation	5
Chapter 3: Existing System and Proposed System	**6**
3.1 Overview of Existing Outpatient Workflows	6
3.2 Drawbacks and Security Vulnerabilities of Current Systems	6
3.3 Proposed MediGuide Care System	6
3.4 Comparative Analysis Matrix	7
Chapter 4: Software and Hardware Requirements	**8**
4.1 Hardware Requirements	8
4.2 Software Requirements and Operating Environments	8
4.3 Runtime Dependencies and Library Specifications	9
Chapter 5: Technology Stack and Architectural Justification	**11**
5.1 Backend: Java 17 LTS and Spring Boot 3.2.5	11
5.2 Persistence: PostgreSQL 16 and Hibernate 6 ORM	11
5.3 Security: Spring Security 6, BCrypt, and JJWT	11
5.4 Email & Notifications: Spring Boot Starter Mail & SHA-256 Tokens	11
5.5 Frontend: React 19, Vite 8, and Modern Vanilla CSS	12
5.6 External AI Integration: Spring WebClient	12

5.7 API Tooling: SpringDoc OpenAPI 3 and Swagger UI 12
Chapter 6: System Analysis and Design 13
6.1 Architectural Principles and Separation of Concerns 13
6.2 Multi-Tier Application Design Pattern 13
6.3 Data Transfer Object (DTO) Boundary Pattern 13
6.4 Concurrency Control & Pessimistic Row Locking Strategy 13
6.5 Data Privacy Model: Segregating Public Clinic vs. Private Personal Information 14
Chapter 7: System Architecture and Workflow Diagrams 16
7.1 Overall System Architecture 16
7.2 Registration, Login, JWT, and Email Verification Workflow 16
7.3 Doctor Availability and Patient Appointment Booking Workflow 18
7.4 Clinical Consultation, Prescription, and Follow-Up Workflow 18
7.5 AI-Assisted Clinical History Summarization Workflow and Safeguards 20
Chapter 8: Database Design, Entities, Relationships, and Data Flow 21
8.1 Relational Database Entity-Relationship Diagram (ERD) 21
8.2 Entity Dictionary and Schema Definitions 21
8.3 Referential Integrity, Cascade Policies, and Constraints 24
8.4 Data Flow and Transactional Boundaries 24
Chapter 9: Module-Wise Implementation 25
9.1 User Authentication, Role-Based Access Control, and Public IDs 25
9.2 Doctor Profile Management & Location-Based Discovery Search 25
9.3 Availability Calendar Scheduler & Slot Management 25
9.4 Appointment Booking, Concurrency Prevention, and Lifecycle Management 25
9.5 Clinical Consultation Documentation & Digital Prescriptions Drawer 26
9.6 Patient Care Timeline & Historical Consultations 26
9.7 AI-Assisted Clinical History Summarizer & Doctor-in-the-Loop Safeguards 26
9.8 Email Verification System, Cooldown Rate Limits, and Anti-Enumeration 26
9.9 Role-Based Access Control Matrix and Privacy Boundaries 27
Chapter 10: API Design, Security, and Endpoint Documentation 28
10.1 RESTful API Principles and Resource Mapping 28
10.2 Comprehensive Endpoint Inventory 28
10.3 Authentication and Authorization Headers 29
10.4 Error Handling Schema and HTTP Status Code Semantics 30
Chapter 11: Testing Strategy, Test Cases, and Verified Results 31
11.1 Testing Methodology and Multi-Layer Test Architecture 31

11.2 Unit, Service, Repository, and Controller Integration Tests 31
11.3 Concurrency & Pessimistic Lock Verification 31
11.4 Comprehensive Test Suite Execution Summary 31
11.5 Frontend Static Code Linting & Production Build Verification 33
Chapter 12: User Interface and Responsive Design 35
12.1 Design Philosophy, Color Palette, and Medical Theme 35
12.2 Modern Vanilla CSS Design System and Token Architecture 35
12.3 Doctor Portal Workflow and Slide-Over Clinical Drawer 35
12.4 Patient Portal Workflow and Care Timeline 35
12.5 Accessibility, Micro-Animations, and Viewport Adaptability 36
Chapter 13: Limitations and Future Enhancements 37
13.1 Operational Constraints and Current Limitations 37
13.2 Technical Enhancements and Scalability Roadmap 37
Chapter 14: Conclusion 39
14.1 Summary of Academic Project Achievements 39
14.2 Learning Outcomes and Viva Preparation Takeaways 39
References 41
Appendix 42
Appendix A: Environment Configuration Reference 42
Appendix B: Sample API Request and Response Payloads 43
Appendix C: Standard Error Response Schema 43

LIST OF FIGURES

Title	Page No.
Figure 7.1: Overall Multi-Tier System Architecture of MediGuide Care	16
Figure 7.2: User Registration, Login, JWT Authentication, and Email Verification Lifecycle	17
Figure 7.3: Doctor Availability Scheduling and Concurrency-Safe Appointment Booking Workflow	18
Figure 7.4: Clinical Consultation Documentation, Digital Prescriptions, and Care Timeline Lifecycle	19
Figure 7.5: AI-Assisted Clinical History Summarizer Workflow and Multi-Layer Medical Safeguards	20
Figure 8.1: Complete Relational Database Entity-Relationship Diagram (ERD)	21
Figure 9.1: Role-Based Access Control (RBAC) Security Architecture and Privacy Boundaries	27

LIST OF TABLES

Title	Page No.
Table 3.1: Comparative Analysis Between Conventional Systems and MediGuide Care	7
Table 4.1: Hardware Requirements for MediGuide Care	8
Table 4.2: Software Environment Specifications	8
Table 4.3: Core Application Libraries and Dependencies	9
Table 8.1: Comprehensive Relational Database Data Dictionary	21
Table 10.1: Comprehensive MediGuide Care REST API Endpoint Inventory	28
Table 10.2: HTTP Status Code Semantics and Error Mapping	30
Table 11.1: Core Automated Test Scenarios and Verified Execution Results	32
Table A.1: Backend Environment Variables Reference	42

CHAPTER 1: INTRODUCTION

1.1 Project Overview and Domain Context

Modern outpatient healthcare delivery relies heavily on efficient communication, precise appointment scheduling, and accurate record-keeping between clinical practitioners and patients. In many small-to-medium clinical settings and private outpatient practices, operations continue to depend on fragmented manual processes, physical ledger books, unstructured telephone or messaging exchanges, and physical paper prescriptions. These legacy practices introduce severe operational bottlenecks, including scheduling collisions, illegible handwritten treatment instructions, scattered clinical consultation histories, and accidental exposure of private contact details.

MediGuide Care is conceived and developed as an integrated Doctor–Patient Consultation and Follow-up Management System. It provides a secure, multi-tier web application designed to transition outpatient clinics into a streamlined digital operational model. The platform unifies patient doctor discovery, dynamic calendar-based availability planning, concurrency-safe appointment booking, slide-over clinical consultation documentation, structured digital prescriptions, and automated AI-assisted clinical history summarization into an accessible, cohesive interface.

1.2 Purpose and Objectives

The primary purpose of MediGuide Care is to eliminate administrative overhead, prevent scheduling conflicts, and improve the quality and continuity of clinical consultations. The core objectives include:

- **Streamlined Scheduling:** Provide physicians with a calendar-based interface to publish distinct availability intervals and allow verified patients to schedule consultation slots with zero double-booking.
- **Structured Clinical Documentation:** Enable doctors to document symptoms, formal diagnoses, treatment advice, and multi-item digital prescriptions with standardized dosage and frequency attributes.
- **Rigorous Data Security and Privacy:** Implement stateless token-based authentication, cryptographic password hashing, and strict role-based access control, while strictly segregating public clinic details from private personal telephone numbers.

• **AI-Assisted History Synthesis:** Equip attending doctors with automated, chronological summaries of prior patient consultations, structured with explicit medical safety safeguards and reviewable draft disclaimers.

• **Cryptographic Email Verification:** Enforce account authenticity through SHA-256 hashed verification tokens, single-use consumption, anti-enumeration responses, and rate-limiting cooldown protection.

1.3 Target Users and Operating Context

MediGuide Care is tailored to serve two primary user classifications within outpatient clinical ecosystems:

1. **Patients:** Individuals seeking medical consultation who need to search for qualified practitioners by medical specialty and geographic locality (city, area, and postal PIN code), reserve convenient consultation times, review their complete historical treatment timeline, and safeguard their private telephone numbers.

2. **Consulting Doctors:** Licensed medical practitioners who require tools to publish flexible consultation schedules across multi-slot calendars, inspect attending queues, document structured electronic prescriptions, and review previous patient diagnoses via on-demand clinical summaries.

1.4 Organization of the Report

This project report is organized into fourteen comprehensive chapters. Chapter 2 details the problem statement, scope, and motivation. Chapter 3 compares existing manual workflows against the proposed system. Chapter 4 specifies hardware and software requirements. Chapter 5 justifies the chosen technology stack. Chapter 6 covers system analysis, design patterns, and concurrency mechanics. Chapter 7 illustrates architectural workflows through high-resolution diagrams. Chapter 8 provides the complete database design and entity dictionary. Chapter 9 details module-by-module implementation. Chapter 10 documents API contracts and security rules. Chapter 11 presents the automated testing strategy and verified test results (298 passing tests). Chapter 12 analyzes the user interface design. Chapter 13 reviews limitations and future enhancements, and Chapter 14 concludes the report.

CHAPTER 2: PROBLEM STATEMENT, OBJECTIVES, SCOPE, AND MOTIVATION

2.1 Problem Statement and Real-World Clinic Challenges

Outpatient healthcare practices face critical operational challenges stemming from reliance on paper records and ad-hoc communication channels:

- **Scheduling Conflicts & Double-Booking:** Manual appointment books cannot coordinate real-time reservations. When patients book through phone calls or front-desk clerks simultaneously, race conditions result in overlapping slots, extended patient wait times, and physician fatigue.
- **Loss of Longitudinal Medical History:** Paper case sheets and physical prescription slips are easily lost, damaged, or misplaced by patients. When consulting a doctor, past medical histories, drug allergies, and earlier treatment responses are frequently unavailable.
- **Unstructured Handwritten Prescriptions:** Illegible handwriting on prescription pads poses grave clinical risks, including medication dispensing errors, incorrect dosage administration, and ambiguous intake frequency.
- **Privacy Violations & Contact Exposure:** In conventional communication (e.g. WhatsApp or personal phone calls), doctor personal telephone numbers and patient private contact details are exposed, causing boundary violations and spam vulnerabilities.
- **Unverified Bot & Phantom Bookings:** Open booking forms without rigorous email verification permit fictitious account creation and abandoned appointment reservations, depriving genuine patients of consultation opportunities.

2.2 Project Objectives

To decisively overcome these vulnerabilities, MediGuide Care establishes the following engineering goals:

1. Engineer a stateless, secure multi-tier RESTful application adhering to the Controller-Service-Repository architectural pattern.
2. Eliminate sequential enumeration attacks by replacing internal database IDs with collision-resistant public alphanumeric identifiers (PAT-XXXXXX and DOC-XXXXXX).

3. Guarantee zero double-booking under concurrent traffic using database-level pessimistic write locking on ascending user IDs combined with transactional SQL overlap validation.

4. Deliver a responsive single-page web client providing distinct, role-enforced portals for patients and doctors.

5. Provide structured electronic prescriptions and digital consultation records automatically bound to appointment completion.

6. Integrate an automated, sanitized AI clinical history summarizer with doctor-in-the-loop clinical disclosure safeguards.

7. Implement cryptographic, rate-limited email verification to protect high-trust clinic operations.

2.3 Project Scope

In-Scope Capabilities (Implemented and Verified):

• Patient and doctor self-registration with salted BCrypt password hashing.

• Stateless HMAC-SHA256 JWT access token generation, validation, and in-memory client storage.

• Cryptographic email verification using 32-byte Base64URL tokens hashed with SHA-256 and 60-second cooldown rate-limiting.

• Multi-field doctor discovery search filtering by specialty, city, locality/area, and 6-digit Indian PIN code.

• Doctor calendar availability planning with multi-slot creation, interval overlap validation, and soft deactivation.

• Pessimistic row-locked appointment booking preventing double-booking race conditions.

• Full appointment lifecycle state machine: BOOKED → COMPLETED or CANCELLED.

• Slide-over clinical consultation drawer for documenting symptoms, diagnoses, advice, follow-up dates, and multi-item prescriptions.

• Patient chronological care timeline and authorized doctor consultation history lookup.

• OpenAI-compatible AI clinical history summarizer with data anonymization and safety prompt safeguards.

• Strict data isolation: patient private phone numbers and doctor personal phone numbers are inaccessible via public endpoints.

• Automated test suite with 298 passing unit, integration, repository, and concurrency tests.

Explicitly Out-of-Scope Roadmap (Planned for Subsequent Sprints):

• Asynchronous distributed message brokers (RabbitMQ / Kafka) and dead-letter retry queues.

• Real-time video/audio telemedicine consultations (WebRTC).

• SMS gateway delivery and QR-code summary sharing for mobile devices.

• Container orchestration (Docker Compose / Kubernetes) and continuous deployment pipelines (Jenkins).

2.4 Academic and Practical Motivation

Developing MediGuide Care serves as a capstone application of enterprise software engineering principles for the Master of Computer Applications (MCA) curriculum. It provides hands-on mastery of enterprise Java with Spring Boot, modern reactive component architectures with React 19, relational ACID transaction management with PostgreSQL and Hibernate, cryptographic security with JWT and salted hashing, and defensive concurrency engineering under multi-threaded database transactions. Practically, the project addresses real outpatient operational inefficiencies, demonstrating how modern web technologies can elevate healthcare delivery.

CHAPTER 3: EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Overview of Existing Outpatient Workflows

In traditional clinic consultation workflows, scheduling is managed either via physical paper appointment registers or through informal digital channels such as phone calls and direct messaging. When a patient arrives, clinic receptionists manually record patient details, retrieve physical paper folders, and direct patients to waiting areas without transparent queue timing. During consultations, physicians handwrite clinical findings, diagnoses, and medication prescriptions on paper slips. Patients retain these slips, while clinics store brief handwritten ledger entries. Follow-up scheduling is typically handled verbally, with patients instructed to return after a specified number of days.

3.2 Drawbacks and Security Vulnerabilities of Current Systems

Conventional clinic consultation models suffer from fundamental architectural and operational vulnerabilities:

- **High Vulnerability to Race Conditions:** Simultaneous bookings taken via phone and in-person front desks frequently collide, resulting in embarrassing double-bookings and prolonged patient disputes.
- **Zero Data Integrity and Loss:** Physical prescription pads and case records are subject to physical wear, water damage, fire risk, and misplacement, permanently severing historical medical continuity.
- **Severe Prescription Ambiguity:** Illegible medical handwriting contributes significantly to pharmacological errors in pharmacies, with patients receiving incorrect drug molecules or dosages.
- **Absence of Role-Based Privacy:** When clinics share doctor phone numbers openly or store patient contact numbers in public spreadsheets, personal contact numbers are exposed to unauthorized marketing or harassment.
- **No Longitudinal Synthesis:** When examining a patient with recurring complaints, physicians have no automated mechanism to review historical diagnoses, forcing repetitive oral questioning during limited consultation windows.

3.3 Proposed MediGuide Care System

MediGuide Care replaces manual and ad-hoc practices with an integrated, secure, multi-tier digital architecture. Physicians establish defined consultation availability intervals through an intuitive month/day visual calendar planner. Verified patients search for certified physicians across medical specialties and local geographies, browse real-time available slots, and execute guaranteed conflict-free reservations coordinated through database pessimistic row-level locking. Attending physicians document structured observations and digital multi-item prescriptions via a slide-over consultation drawer, which automatically marks the appointment as completed and publishes the consultation record to the patient's care timeline. Furthermore, physicians can invoke an AI clinical summarizer that sanitizes past records and synthesizes a structured review draft with doctor-in-the-loop safeguards.

3.4 Comparative Analysis Matrix

Table 3.1 highlights the direct operational and architectural differences between conventional clinic systems and MediGuide Care.

Dimension / Feature	Existing Conventional System	Proposed MediGuide Care System
Scheduling Method	Physical paper ledgers or unstructured phone calls	Calendar-based multi-slot planner with real-time UI
Concurrency Defense	None; frequent overlapping double-bookings	Pessimistic row-locking on ascending user IDs + range checks
Prescription Delivery	Handwritten paper slips; illegible handwriting risks	Structured digital prescriptions with dosage, frequency, duration
Medical History Access	Scattered paper files; frequently lost by patients	Permanent chronological care timeline accessible via portal
User Identification	Plain sequential integers; vulnerable to enumeration	Obfuscated public alphanumeric IDs (DOC- / PAT-XXXXXX)
Authentication & Auth	Unsecured spreadsheets or basic session passwords	Stateless HMAC-SHA256 JWT + salted BCrypt hashing + RBAC
Account Verification	Unverified phone numbers or open web forms	Cryptographic SHA-256 tokens + 60s cooldown rate limiting
Clinical History Review	Manual page-turning of thick physical folders	AI-assisted structured draft summary with safety safeguards
Contact Privacy	Uncontrolled disclosure of private phone numbers	Strict segregation: public clinic phone vs. private personal phone

Table 3.1: *Comparative Analysis Between Conventional Systems and MediGuide Care*

CHAPTER 4: SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Hardware Requirements

To ensure reliable execution, compilation, and concurrent transaction management, the development, server, and client environments must satisfy the minimum specifications outlined in Table 4.1.

Environment	Component	Minimum Requirement	Recommended Specification
Developer Workstation	Processor	Dual-Core x86_64 / ARM (2.0 GHz)	Quad-Core Intel i5/i7 or AMD Ryzen 5/7 (3.0+ GHz)
Developer Workstation	RAM	8 GB DDR4	16 GB DDR4/DDR5
Developer Workstation	Storage	20 GB available SSD space	50 GB NVMe M.2 Solid State Drive
Backend Server Host	Processor	1 vCPU (2.4 GHz)	2+ vCPUs dedicated
Backend Server Host	RAM	2 GB RAM (JVM heap: 1024 MB)	4 GB+ RAM (JVM heap: 2048 MB)
Backend Server Host	Network	10 Mbps continuous bandwidth	100 Mbps / 1 Gbps low-latency interface
Database Server	RAM & Disk	2 GB RAM, 10 GB SSD for tables/WAL	4 GB RAM, High-IOPS NVMe SSD storage
Client Devices	Processor & RAM	1.5 GHz Dual-Core, 2 GB RAM	2.0 GHz Quad-Core, 4 GB+ RAM (Desktop/Tablet)

Table 4.1: *Hardware Requirements for MediGuide Care*

4.2 Software Requirements and Operating Environments

The software components, runtime virtual machines, and development tools required for the project are summarized in Table 4.2.

Category	Software / Framework	Version Tested	Role / Operational Purpose
Operating System	Windows 10/11 / Ubuntu Linux	Windows 11 (64-bit)	Workstation and host operating platform

Category	Software / Framework	Version Tested	Role / Operational Purpose
JDK Runtime	Eclipse Temurin OpenJDK / Oracle JDK	Java SE 17.0.7 (LTS)	Java Virtual Machine compiler and runtime
Build Tool (Backend)	Apache Maven	Maven 3.8.x / 3.9.x	Dependency resolution, compilation, test runner
Relational Database	PostgreSQL Server	PostgreSQL 16.x	Production ACID relational data storage
In-Memory Database	H2 Database Engine	H2 2.2.x (test-scope)	Isolated automated unit & integration testing
Node Runtime (UI)	Node.js & npm package manager	Node 18+ (npm 9+)	Frontend development server and bundle compiler
Frontend Bundler	Vite Build Tool	Vite 8.3.4	Fast Hot Module Replacement (HMR) and bundling
Source Code Editor	Visual Studio Code / IntelliJ IDEA	VS Code & IntelliJ 2024	Development, debugging, and git integration

Table 4.2: *Software Environment Specifications*

4.3 Runtime Dependencies and Library Specifications

Key third-party dependencies defined in the Maven pom.xml and Node package.json include:

Library / Dependency	Group / Scope	Artifact Version	Functional Responsibility
spring-boot-starter-web	org.springframework.boot	3.2.5	Embedded Tomcat, DispatcherServlet, REST APIs
spring-boot-starter-data-jpa	org.springframework.boot	3.2.5	Spring Data JPA, Hibernate 6 ORM, HikariCP
postgresql	org.postgresql (runtime)	42.6.2	PostgreSQL JDBC Type-4 connection driver
spring-boot-starter-security	org.springframework.boot	3.2.5	SecurityFilterChain, authentication, filters
jjwt-api / impl / jackson	io.jsonwebtoken	0.12.5	HMAC-SHA256 JWT token encoding, parsing, claims
spring-boot-starter-mail	org.springframework.boot	3.2.5	JavaMailSender, Jakarta Mail SMTP transport

Library / Dependency	Group / Scope	Artifact Version	Functional Responsibility
springdoc-openapi-starter	org.springdoc	2.5.0	OpenAPI v3 schema generator and Swagger UI
spring-boot-starter-validation	org.springframework.boot	3.2.5	Hibernate Validator runtime (@NotBlank, @Email)
react & react-dom	npm dependency	19.0.0	Declarative UI rendering and modular state hooks
oxlint	npm devDependency	Latest	High-performance JavaScript static code linter

Table 4.3: *Core Application Libraries and Dependencies*

CHAPTER 5: TECHNOLOGY STACK AND ARCHITECTURAL JUSTIFICATION

5.1 Backend: Java 17 LTS and Spring Boot 3.2.5

Java 17 was selected as the foundational runtime because it is a Long-Term Support (LTS) release offering high stability, strong performance enhancements, and modern language features such as records, text blocks, pattern matching for switch, and sealed classes. **Spring Boot 3.2.5** leverages the Jakarta EE 10 baseline, delivering an production-ready application framework. It eliminates boilerplate XML configuration, embeds a high-throughput Apache Tomcat web container, and provides dependency injection enabling strict decoupling between controller, service, and repository tiers.

5.2 Persistence: PostgreSQL 16 and Hibernate 6 ORM

PostgreSQL 16 was selected for persistence due to its enterprise-grade ACID compliance, robust indexing capabilities, and advanced transactional isolation levels. In healthcare scheduling, race conditions must be coordinated deterministically. PostgreSQL natively supports row-level exclusive write locks (SELECT ... FOR UPDATE), which MediGuide Care employs to prevent double-booking. **Spring Data JPA** backed by **Hibernate 6** maps relational tables to domain entities cleanly, manages foreign key constraints, and utilizes HikariCP for high-performance database connection pooling.

5.3 Security: Spring Security 6, BCrypt, and JJWT

Authentication is architected as completely stateless using **Spring Security 6** and **JJWT 0.12.5**. Client requests transmit a compact, cryptographically signed Bearer JWT token in the HTTP Authorization header. Tokens are deterministically signed using HMAC-SHA256 with a high-entropy secret key of at least 256 bits. User passwords are never stored in plaintext; instead, they are hashed using the **BCrypt** salted key-derivation function configured in PasswordConfig. Fine-grained method security (@PreAuthorize) ensures that patients cannot invoke doctor scheduling endpoints and doctors cannot manipulate unassigned patient records.

5.4 Email & Notifications: Spring Boot Starter Mail & SHA-256 Tokens

Account authenticity is guaranteed via **Spring Boot Starter Mail** (JavaMailSender) over configurable SMTP. Rather than persisting raw verification tokens in the database, MediGuide Care generates a cryptographically random 32-byte Base64URL string, computes its 64-character SHA-256 hash, and stores only the hash in email_verification_tokens. Tokens expire automatically after 24 hours and are deleted immediately upon single-use verification. A 60-second rate-limiting cooldown prevents email flooding, and uniform responses safeguard against user enumeration.

5.5 Frontend: React 19, Vite 8, and Modern Vanilla CSS

The client layer is implemented as a Single-Page Application (SPA) using **React 19** and **Vite 8**. React's component model coordinates dynamic portal views (DoctorPortal, PatientPortal, DoctorAvailabilityCalendar, DoctorClinicalDrawer). Vite provides sub-second Hot Module Replacement (HMR) and optimized rollup production bundles (341 kB minified JS). The interface avoids bulky CSS utility frameworks, instead employing a custom **Vanilla CSS design system** defined in index.css and RolePortal.css. It utilizes semantic design tokens, clean glassmorphism accents, and accessible typography (Google Fonts Outfit and Inter) tailored for medical clinic environments.

5.6 External AI Integration: Spring WebClient

The clinical history summarizer integrates with OpenAI-compatible large language model APIs via a non-blocking WebClient wrapper (OpenAiCompatibleClinicalSummaryClient). The integration is strictly isolated: prompts contain only chronological consultation data with internal keys stripped, subject to strict medical safety system instructions and bounded token budgets. In test environments, deterministic mocks prevent outbound HTTP network calls, guaranteeing reliable builds.

5.7 API Tooling: SpringDoc OpenAPI 3 and Swagger UI

Comprehensive API documentation is generated dynamically at runtime using **springdoc-openapi 2.5.0**. The interactive Swagger UI (accessible at /swagger-ui.html) and OpenAPI JSON schema (/api-docs) allow developers and evaluators to inspect schema definitions, review request parameters, and execute authenticated API calls by supplying the JWT Bearer token directly through the UI authorize modal.

CHAPTER 6: SYSTEM ANALYSIS AND DESIGN

6.1 Architectural Principles and Separation of Concerns

MediGuide Care strictly adheres to core software engineering principles: Separation of Concerns (SoC), Single Responsibility Principle (SRP), and the Principle of Least Privilege. The codebase is decoupled into distinct vertical tiers: the client layer manages user interaction and token storage; the security gateway authenticates incoming HTTP requests; controllers validate DTO syntax; services execute business logic and concurrency locks; and repositories manage database transactions. No database queries exist in controllers, and no presentation markup exists in backend services.

6.2 Multi-Tier Application Design Pattern

The backend follows the established enterprise **Controller → Service → Repository** architectural pattern:

1. **REST Controllers:** Accept incoming HTTP requests, unpack JSON bodies into typed DTOs, invoke Jakarta Bean Validation (@Valid), and return standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 409 Conflict).
2. **Service Layer:** Contains all business rules, role authorization checks, pessimistic concurrency locks, cryptographic token generation, and transactional boundaries (@Transactional).
3. **Repository Layer:** Extends JpaRepository to execute parameterized SQL and JPQL queries against PostgreSQL, managing entity lifecycles and relationship fetching.

6.3 Data Transfer Object (DTO) Boundary Pattern

Domain entities are never exposed directly to external HTTP clients. MediGuide Care enforces a strict DTO boundary pattern to prevent accidental data leakage, cyclic JSON serialization, and mass-assignment vulnerabilities. Separate request DTOs (e.g., RegisterRequest, BookAppointmentRequest, CreateConsultationRequest) and response DTOs (e.g., AuthResponse, AppointmentResponse, PublicDoctorProfileResponse) ensure that passwords, internal database auto-increment keys, and confidential phone numbers are filtered out before transmission.

6.4 Concurrency Control & Pessimistic Row Locking Strategy

Preventing appointment double-booking under concurrent client requests is a critical engineering requirement. In a naive implementation, checking slot availability and subsequently inserting an appointment introduces a race condition: two concurrent threads can both observe the slot as free, and both will proceed to insert conflicting records.

MediGuide Care solves this deterministically through a multi-stage concurrency coordination algorithm in AppointmentService:

1. **Deterministic Lock Acquisition Order:** The service identifies the IDs of both the patient and the doctor. To prevent distributed deadlocks where Thread A locks Patient then Doctor while Thread B locks Doctor then Patient, locks are **always acquired in ascending User ID order** (min(patientId, doctorId) followed by max(patientId, doctorId)).

2. **Pessimistic Row-Level Write Locks:** The service executes userRepository.findByIdForUpdate(id), issuing SQL SELECT ... FOR UPDATE. This serializes concurrent booking requests for either the doctor or the patient at the database level.

3. **Transactional Entity Refresh:** Under the active lock, the doctor availability entity is refreshed from the database (entityManager.refresh(availability)) to observe any concurrent schedule deactivations.

4. **SQL Overlap Range Query:** The engine queries for existing appointments in BOOKED status overlapping the requested interval using half-open time interval logic: WHERE doctor_id = :doctorId AND status = 'BOOKED' AND start_time < :newEndTime AND end_time > :newStartTime.

5. **Conflict Resolution:** If any conflict is detected, the transaction aborts and returns an HTTP 409 Conflict with an explanatory message; otherwise, the appointment is created atomically.

6.5 Data Privacy Model: Segregating Public Clinic vs. Private Personal Information

MediGuide Care implements a robust three-tier data privacy model that eliminates unauthorized disclosure of personal phone numbers:

• **Patient Contact Number (users.phone_number):** Strictly private. Accessible and editable exclusively by the authenticated patient via GET/PUT /api/v1/patient/profile. It is completely excluded from doctor search results, public lookup endpoints, and unauthenticated views.

• **Doctor Clinic Phone (doctor_profiles.clinic_phone):** Public operational contact. Displayed on public doctor profile cards and search results so prospective patients can make clinic inquiries.

• **Doctor Personal Phone (doctor_profiles.personal_phone):** Strictly confidential. Returned only to the authenticated doctor in DoctorProfileResponse when managing their own profile. It is completely stripped from PublicDoctorProfileResponse and public search results.

• **Public MediGuide IDs (DOC-XXXXXX / PAT-XXXXXX):** Replaces internal database integer IDs on all client responses, preventing attackers from probing or enumerating total clinic registration counts.

CHAPTER 7: SYSTEM ARCHITECTURE AND WORKFLOW DIAGRAMS

7.1 Overall System Architecture

MediGuide Care is architected as a decoupled, multi-tier distributed web application designed for high availability, strict data boundary enforcement, and maintainability. Figure 7.1 illustrates the structural interaction across all application layers, including the React 19 Single-Page Application (SPA) client, the Spring Security 6 gateway, the Spring Boot REST controllers, the core business service layer, the PostgreSQL relational persistence layer, and external integration points (SMTP and AI summarizer).

Figure 7.1: Overall Multi-Tier System Architecture of MediGuide Care

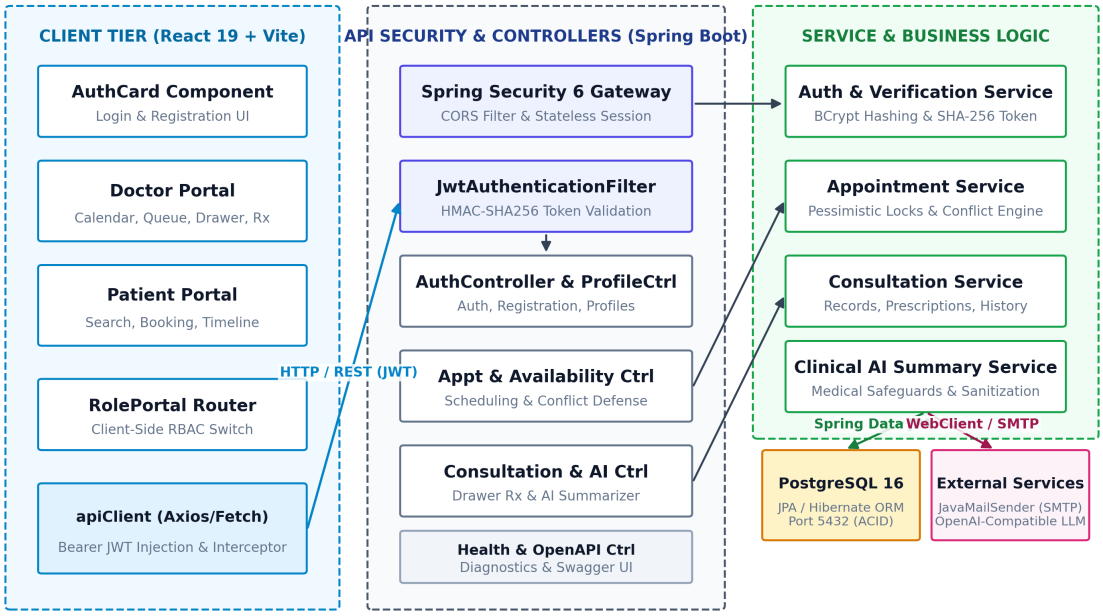


Figure 7.1: *Overall Multi-Tier System Architecture of MediGuide Care*

7.2 Registration, Login, JWT, and Email Verification Workflow

User onboarding and authentication follow a multi-stage security lifecycle depicted in Figure 7.2. Upon registration, user passwords are salted and hashed using BCrypt. The system assigns a unique public identifier (PAT-XXXXXX for patients or DOC-XXXXXX for doctors) and persists the record with email_verified = false. Simultaneously, a 32-byte cryptographically secure random token is generated. Only its 64-character SHA-256 hash is stored in email_verification_tokens, while the plaintext token is transmitted in a verification hyperlink via SMTP. Upon clicking the link, the user's account is validated and the token is consumed. Stateless logins issue an HMAC-SHA256 signed JWT containing the user's email, role, public ID, and verification status, which the frontend stores in-memory.

Figure 7.2: Registration, Login, JWT Authentication & Email Verification Lifecycle

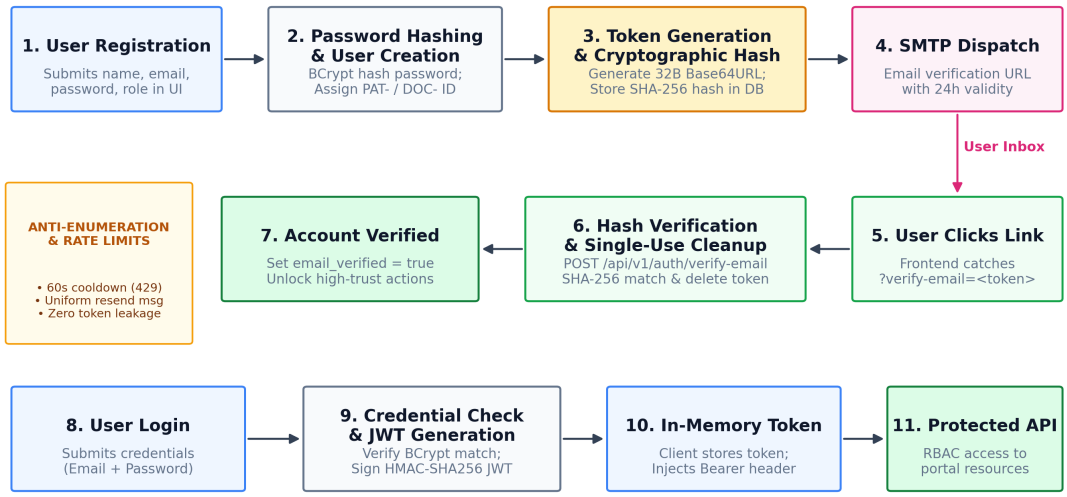


Figure 7.2: *User Registration, Login, JWT Authentication, and Email Verification Lifecycle*

7.3 Doctor Availability and Patient Appointment Booking Workflow

Figure 7.3 details the dual-lane workflow governing doctor schedule creation and patient appointment reservations. A verified doctor publishes availability intervals via the calendar scheduler. The backend validates that the start time precedes the end time, ensures the interval is in the future, and verifies that no overlapping availability window exists for that doctor. Prospective patients discover active slots through multi-field search. During booking, AppointmentService acquires exclusive pessimistic row-level write locks on both the patient and doctor records in ascending User ID order, executes a transactional SQL overlap range query, and atomically creates the appointment in BOOKED status, preventing concurrent double-booking.

Figure 7.3: Doctor Availability Publishing & Concurrency-Safe Appointment Booking

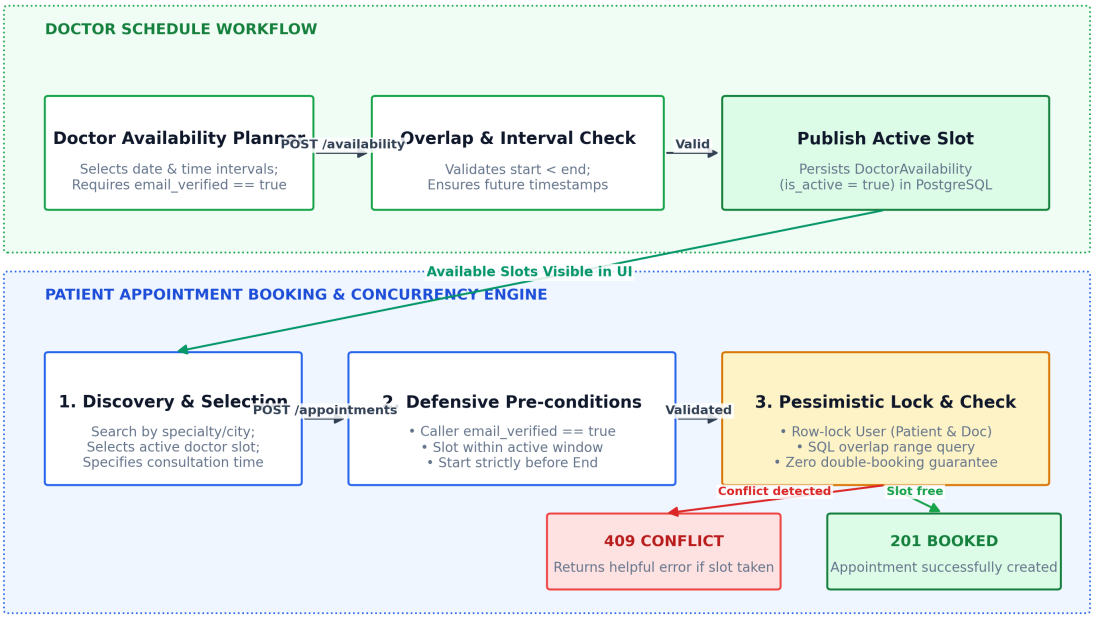


Figure 7.3: *Doctor Availability Scheduling and Concurrency-Safe Appointment Booking Workflow*

7.4 Clinical Consultation, Prescription, and Follow-Up Workflow

The consultation lifecycle shown in Figure 7.4 connects appointment scheduling with clinical documentation. From the attending queue, the doctor selects a patient in BOOKED status and opens the slide-over consultation drawer. The doctor inputs clinical observations, symptoms, medical diagnosis, treatment advice, and recommended follow-up date. Additionally, multiple digital prescription items (medication name, dosage, frequency, duration) can be attached. Submitting the drawer initiates an atomic database transaction that persists the ConsultationRecord and Prescription entities, and transitions the appointment status from BOOKED to COMPLETED. The record immediately becomes accessible on the patient's chronological care timeline and the doctor's historical consultation registry.

Figure 7.4: Clinical Consultation Documentation, Digital Prescriptions & Timeline Lifecycle

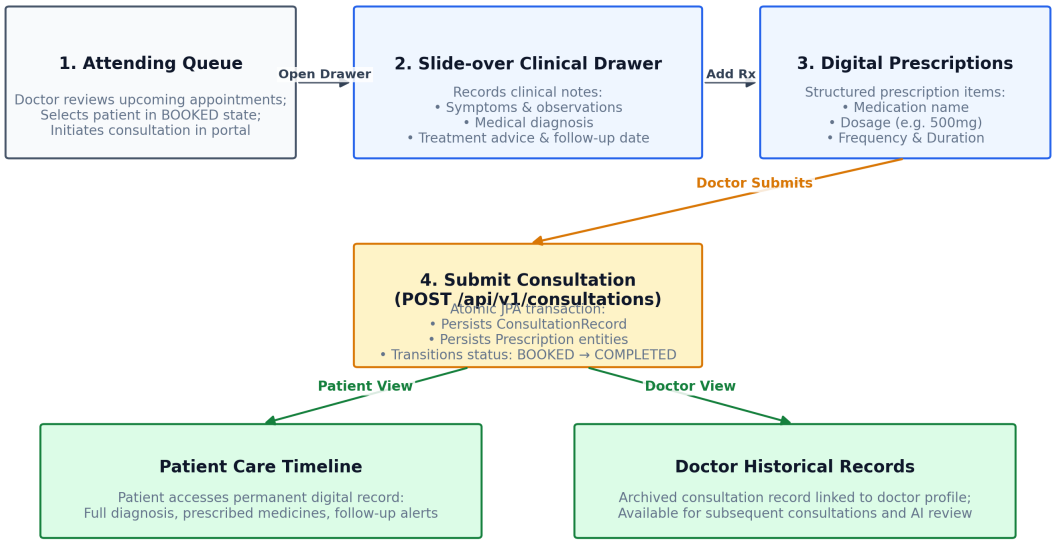


Figure 7.4: *Clinical Consultation Documentation, Digital Prescriptions, and Care Timeline Lifecycle*

7.5 AI-Assisted Clinical History Summarization Workflow and Safeguards

Figure 7.5 illustrates the multi-tier safeguards governing the AI clinical history summarizer. When an attending doctor requests an AI summary for a patient, the frontend requires the doctor to confirm an explicit clinical disclosure acknowledgement. The backend verifies that the doctor is actively authorized to consult the patient, retrieves past consultations, strips all internal database keys and personal contact data, and bounds the payload within a strict character budget (max 6,000 characters). The payload is dispatched alongside a strict medical safety system prompt that forbids inferring new diagnoses or changing drug dosages. The returned response is explicitly labeled as an unverified draft for physician review, with copy-to-clipboard functionality enabling the doctor to verify and incorporate relevant excerpts into the consultation drawer.

Figure 7.5: AI-Assisted Clinical History Summarizer Workflow & Multi-Layer Medical Safeguards

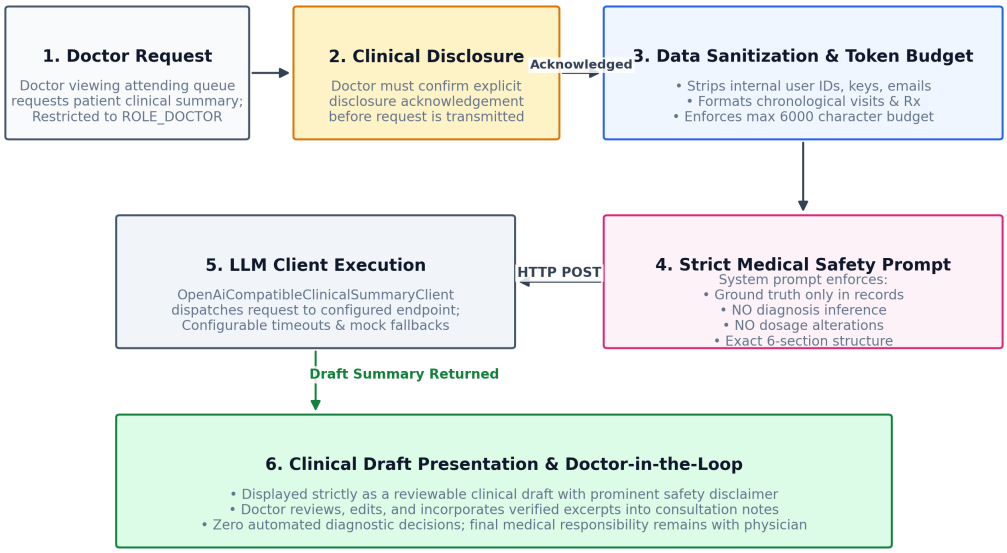


Figure 7.5: *AI-Assisted Clinical History Summarizer Workflow and Multi-Layer Medical Safeguards*

CHAPTER 8: DATABASE DESIGN, ENTITIES, RELATIONSHIPS, AND DATA FLOW

8.1 Relational Database Entity-Relationship Diagram (ERD)

The persistence layer of MediGuide Care is modeled around seven relational tables in PostgreSQL 16. Figure 8.1 depicts the complete Entity-Relationship Diagram (ERD), showing primary keys (PK), foreign keys (FK), unique constraints (UK), and cardinalities across all domain entities.

Figure 8.1: Complete Relational Database Entity-Relationship Diagram (ERD)

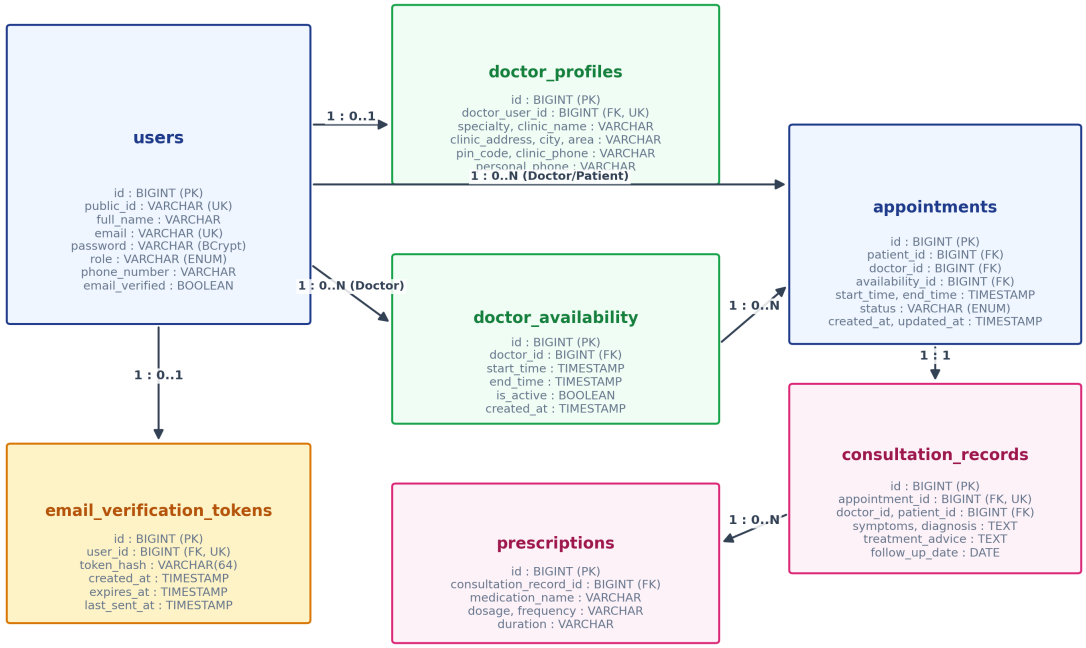


Figure 8.1: *Complete Relational Database Entity-Relationship Diagram (ERD)*

8.2 Entity Dictionary and Schema Definitions

Table 8.1 provides the comprehensive data dictionary detailing column names, data types, constraints, and business descriptions for every table in the PostgreSQL database.

Table Name	Column Name	Data Type	Constraints	Description / Purpose
users	id	BIGINT	PK, AUTO_INC	Internal surrogate primary key

Table Name	Column Name	Data Type	Constraints	Description / Purpose
users	public_id	VARCHAR(30)	UK, NOT NULL	Public ID (PAT- / DOC-XXXXXX)
users	full_name	VARCHAR(100)	NOT NULL	User full name for display
users	email	VARCHAR(120)	UK, NOT NULL	Normalized login email address
users	password	VARCHAR(255)	NOT NULL	Salted BCrypt password hash
users	role	VARCHAR(20)	NOT NULL	Enum: PATIENT or DOCTOR
users	phone_number	VARCHAR(30)	NULLABLE	Private patient contact telephone
users	email_verified	BOOLEAN	NOT NULL, DEF: TRUE	Account email confirmation flag
doctor_profiles	id	BIGINT	PK, AUTO_INC	Primary key for doctor profile
doctor_profiles	doctor_user_id	BIGINT	FK (users), UK	One-to-one link to doctor user
doctor_profiles	specialty	VARCHAR(100)	NULLABLE	Medical specialty (e.g. Cardiology)
doctor_profiles	clinic_name	VARCHAR(150)	NULLABLE	Physical clinic/hospital name
doctor_profiles	city, area	VARCHAR(100)	NULLABLE	Geographic municipality and locality
doctor_profiles	pin_code	VARCHAR(10)	NULLABLE	Validated 6-digit Indian postal code
doctor_profiles	clinic_phone	VARCHAR(30)	NULLABLE	Public clinic contact for inquiries
doctor_profiles	personal_phone	VARCHAR(30)	NULLABLE	Strictly private doctor telephone
doctor_availability	id	BIGINT	PK, AUTO_INC	Availability slot identifier
doctor_availability	doctor_id	BIGINT	FK (users)	Many-to-one link to doctor

Table Name	Column Name	Data Type	Constraints	Description / Purpose
doctor_availability	start_time, end_time	TIMESTAMP	NOT NULL	Consultation time interval bounds
doctor_availability	is_active	BOOLEAN	NOT NULL, DEF: TRUE	Active discovery flag for patients
appointments	id	BIGINT	PK, AUTO_INC	Appointment record identifier
appointments	patient_id	BIGINT	FK (users)	Many-to-one link to booking patient
appointments	doctor_id	BIGINT	FK (users)	Many-to-one link to attending doctor
appointments	availability_id	BIGINT	FK (availability)	Slot within which booking falls
appointments	start_time, end_time	TIMESTAMP	NOT NULL	Reserved consultation time bounds
appointments	status	VARCHAR(20)	NOT NULL	Enum: BOOKED, COMPLETED, CANCELLED
consultation_records	id	BIGINT	PK, AUTO_INC	Clinical consultation record key
consultation_records	appointment_id	BIGINT	FK (appts), UK	One-to-one link to appointment
consultation_records	doctor_id, patient_id	BIGINT	FK (users)	Attending physician and patient
consultation_records	symptoms, diagnosis	TEXT	NULLABLE	Recorded clinical observations
consultation_records	treatment_advice	TEXT	NULLABLE	Doctor's medical instructions
consultation_records	follow_up_date	DATE	NULLABLE	Recommended next consultation date
prescriptions	id	BIGINT	PK, AUTO_INC	Prescription line item key
prescriptions	consultation_record_id	BIGINT	FK (consultation)	Parent consultation record link
prescriptions	medication_name	VARCHAR(150)	NOT NULL	Prescribed drug / molecule name

Table Name	Column Name	Data Type	Constraints	Description / Purpose
prescriptions	dosage, frequency	VARCHAR(100)	NULLABLE	Standardized dosage & timing
prescriptions	duration	VARCHAR(50)	NULLABLE	Prescription treatment duration
email_verification_tokens	id	BIGINT	PK, AUTO_INC	Verification token primary key
email_verification_tokens	user_id	BIGINT	FK (users), UK	One-to-one link to user account
email_verification_tokens	token_hash	VARCHAR(64)	NOT NULL	SHA-256 cryptographic token hash
email_verification_tokens	expires_at, last_sent_at	TIMESTAMP	NOT NULL	Expiry & cooldown rate tracking

Table 8.1: *Comprehensive Relational Database Data Dictionary*

8.3 Referential Integrity, Cascade Policies, and Constraints

Foreign key constraints enforce referential integrity across all related tables. In ConsultationRecord, appointment_id carries a strict UNIQUE constraint, guaranteeing that an appointment can yield at most one clinical consultation record. A cascade policy (CascadeType.ALL, orphanRemoval = true) is applied from ConsultationRecord to Prescription, ensuring that all prescribed medications are persisted and cleaned up atomically alongside the parent record. Similarly, EmailVerificationToken has a user_id UNIQUE constraint, ensuring that at most one active verification token exists per account.

8.4 Data Flow and Transactional Boundaries

Every write operation across the service layer is executed within an atomic Spring @Transactional boundary. In appointment booking, locking both user rows, verifying slot validity, and executing the conflict check occur within a single transaction; if an overlap is detected, the transaction rolls back cleanly with zero partial state committed. In consultation submission, persisting symptoms, diagnosis, prescription items, and updating the appointment status to COMPLETED occur atomically, guaranteeing complete consistency between appointments and consultation histories.

CHAPTER 9: MODULE-WISE IMPLEMENTATION

9.1 User Authentication, Role-Based Access Control, and Public IDs

The authentication module (AuthController, AuthService) manages registration and login for patients and doctors. Self-registration as an administrator is strictly prohibited. Passwords are encrypted using BCrypt in PasswordConfig. To prevent sequential database enumeration, PublicIdGenerator assigns collision-resistant IDs: DOC-XXXXXX for doctors and PAT-XXXXXX for patients using SecureRandom. To ensure backward compatibility with legacy databases, PublicIdBackfillRunner automatically executes on startup to populate public IDs for any legacy accounts lacking one.

9.2 Doctor Profile Management & Location-Based Discovery Search

Doctors configure their medical specialty, clinic name, physical address, municipality/city, area, 6-digit postal PIN code, clinic phone, and private personal phone via DoctorProfileController. The public search engine (GET /api/v1/doctors) allows patients to search by specialty, city, area, and PIN code. The underlying repository query utilizes case-insensitive substring matching for text fields and exact matching for PIN codes, combining active criteria using boolean AND logic. Search responses return PublicDoctorProfileResponse, which displays clinic contact details while strictly suppressing personal doctor phone numbers and login emails.

9.3 Availability Calendar Scheduler & Slot Management

The availability module (DoctorAvailabilityController, DoctorAvailabilityService) empowers doctors to plan consultation hours. In the frontend, DoctorAvailabilityCalendar provides an intuitive visual month and day grid. Doctors can define multiple morning, afternoon, or evening slots. Backend validation ensures start times precede end times, slots reside in the future, and new intervals do not overlap existing active slots for that doctor. Doctors can deactivate unbooked availability slots, which immediately hides them from patient discovery.

9.4 Appointment Booking, Concurrency Prevention, and Lifecycle Management

Managed by AppointmentController and AppointmentService, booking allows patients to reserve consultation intervals. Defensive validation ensures the patient's email is verified, the requested time falls within the doctor's active availability window, and start time precedes end time. Concurrency is handled via pessimistic write locking on user IDs in ascending order, followed by SQL range conflict checks. The appointment lifecycle supports status transitions from BOOKED to COMPLETED (upon consultation recording) or CANCELLED (prior to consultation by either party).

9.5 Clinical Consultation Documentation & Digital Prescriptions Drawer

When consulting a patient, the doctor opens the slide-over DoctorClinicalDrawer. The doctor records clinical observations, symptoms, diagnosis, treatment advice, and recommended follow-up date. Standardized digital prescription items (medication name, dosage, frequency, duration) are added dynamically. Submitting the consultation atomically stores the records and updates the appointment status to COMPLETED.

9.6 Patient Care Timeline & Historical Consultations

Patients can inspect their complete medical history via GET /api/v1/consultations/history/patient. The timeline chronologically displays past consultations, diagnoses, prescribed medications, and follow-up guidance. Authorized doctors can also review historical consultation records for patients who have active or past appointments with that doctor, fostering informed clinical continuity.

9.7 AI-Assisted Clinical History Summarizer & Doctor-in-the-Loop Safeguards

Implemented in ClinicalHistorySummaryService, the AI summarizer synthesizes patient consultation histories into structured review drafts. Medical safety is enforced at multiple levels: (1) doctors must explicitly confirm a disclosure acknowledgement; (2) prompts contain strictly chronological clinical data with personal identifiers stripped; (3) character limits guard against prompt injection; (4) system prompts prohibit inferring new diagnoses or changing dosages; and (5) output is presented strictly as a reviewable clinical draft with prominent safety disclaimers.

9.8 Email Verification System, Cooldown Rate Limits, and Anti-Enumeration

Powered by EmailVerificationService, the email subsystem uses 32-byte Base64URL random tokens hashed with SHA-256 in email_verification_tokens. Tokens expire in 24 hours and are permanently deleted upon single use. A 60-second cooldown rate limit returns HTTP 429 with a Retry-After header if triggered. Uniform messaging for non-existent or verified emails prevents account enumeration attacks. Unverified accounts have progressive access: they can log in to view read-only dashboards, but high-trust actions (appointment booking and schedule publishing) require email verification.

9.9 Role-Based Access Control Matrix and Privacy Boundaries

Figure 9.1 visualizes the complete Role-Based Access Control (RBAC) permission matrix, highlighting public access, patient capabilities, doctor privileges, and strict privacy boundaries.

Figure 9.1: Role-Based Access Control (RBAC) Security Architecture & Privacy Boundaries

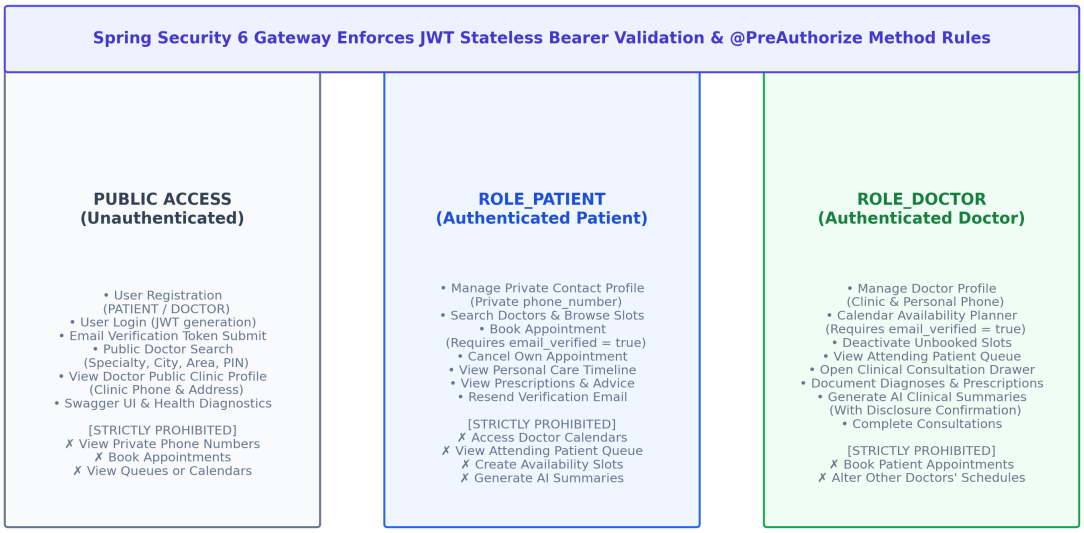


Figure 9.1: *Role-Based Access Control (RBAC) Security Architecture and Privacy Boundaries*

CHAPTER 10: API DESIGN, SECURITY, AND ENDPOINT DOCUMENTATION

10.1 RESTful API Principles and Resource Mapping

MediGuide Care's API is designed in strict compliance with REST architectural constraints. Resources are addressed using plural nouns (e.g., /api/v1/doctors, /api/v1/appointments, /api/v1/consultations). Standard HTTP verbs denote operations: GET for retrieval, POST for creation, and PUT for idempotent updates. All request payloads and responses use UTF-8 JSON. Every response includes standard HTTP status codes and structured response envelopes.

10.2 Comprehensive Endpoint Inventory

Table 10.1 provides the complete inventory of all REST endpoints exposed by the Spring Boot backend.

HTTP Method	Endpoint Path	Access Role	Functional Description
POST	/api/v1/auth/register	Public	Register patient or doctor account with BCrypt password
POST	/api/v1/auth/login	Public	Authenticate user and issue HMAC-SHA256 JWT access token
POST	/api/v1/auth/verify-email	Public	Validate token hash, verify user, delete token
POST	/api/v1/auth/resend-verification	Public	Rate-limited (60s cooldown) verification token resend
GET	/api/v1/doctors	Public	Search doctors by specialty, city, area, or PIN code
GET	/api/v1/doctors/{publicId}	Public	Public doctor profile details (clinic phone only)
GET	/api/v1/doctor/profile	ROLE_DOCTOR	Fetch authenticated doctor's full profile (with personal phone)
PUT	/api/v1/doctor/profile	ROLE_DOCTOR	Update doctor profile and clinic/personal phone numbers
GET	/api/v1/patient/profile	ROLE_PATIENT	Fetch authenticated patient's private contact profile

HTTP Method	Endpoint Path	Access Role	Functional Description
PUT	/api/v1/patient/profile	ROLE_PATIENT	Update authenticated patient's private phone number
POST	/api/v1/doctor/availability	ROLE_DOCTOR	Publish consultation availability slots (verified only)
GET	/api/v1/doctor/availability	ROLE_DOCTOR	List all availability slots scheduled by authenticated doctor
DELETE	/api/v1/doctor/availability/{id}	ROLE_DOCTOR	Deactivate unbooked doctor availability slot
GET	/api/v1/doctor/availability/active	ROLE_PATIENT	Browse active published slots for a given doctor
POST	/api/v1/appointments	ROLE_PATIENT	Book appointment with concurrency locks (verified only)
GET	/api/v1/appointments/patient	ROLE_PATIENT	List all consultation appointments booked by patient
GET	/api/v1/appointments/doctor	ROLE_DOCTOR	List all upcoming attending appointments for doctor
PATCH	/api/v1/appointments/{id}/cancel	PATIENT/DOCTOR	Cancel a booked consultation appointment
POST	/api/v1/consultations	ROLE_DOCTOR	Record consultation, diagnosis, Rx; mark COMPLETED
GET	/api/v1/consultations/history/patient	ROLE_PATIENT	Retrieve personal longitudinal care history timeline
GET	/api/v1/consultations/history/patient/{id}	ROLE_DOCTOR	Retrieve patient consultation history for attending doctor
POST	/api/v1/clinical-summary/patient/{id}	ROLE_DOCTOR	Generate AI clinical history summary with safety safeguards
GET	/api/v1/health	Public	System health check and database connectivity ping

Table 10.1: *Comprehensive MediGuide Care REST API Endpoint Inventory*

10.3 Authentication and Authorization Headers

Protected endpoints require clients to transmit the JWT access token in the standard HTTP Authorization header: Authorization: Bearer <token>. The JwtAuthenticationFilter intercepts the request, verifies the signature against the server's 256-bit secret key, parses claims (subject, role, userId, publicId), and populates Spring's SecurityContextHolder. Unauthenticated requests to protected endpoints trigger CustomAuthenticationEntryPoint (HTTP 401 Unauthorized), while unauthorized role attempts trigger CustomAccessDeniedHandler (HTTP 403 Forbidden).

10.4 Error Handling Schema and HTTP Status Code Semantics

MediGuide Care handles exceptions uniformly through GlobalExceptionHandler, returning structured JSON error bodies containing timestamp, status, error, and message. Table 10.2 details the status code semantics.

HTTP Status	Error Classification	Domain Trigger / Scenario	Client Remediation
200 OK	Success	Resource retrieved or updated successfully	Process response payload
201 Created	Resource Created	User registered, appointment booked, consultation saved	Record created successfully
400 Bad Request	Validation Error	Invalid date format, end time before start time, past time	Correct input fields and retry
401 Unauthorized	Authentication Required	Missing, invalid, expired, or tampered JWT token	Log in again to obtain fresh JWT
403 Forbidden	Access Denied	Patient invoking doctor API or doctor viewing unassigned patient	Verify permissions for caller role
404 Not Found	Resource Not Found	Doctor, slot, or appointment ID does not exist	Check identifier in URL request path
409 Conflict	Concurrency Conflict	Appointment slot already booked or availability overlap	Select a different time interval
429 Too Many Requests	Rate Limit Exceeded	Resend verification requested within 60s cooldown	Wait for Retry-After duration

Table 10.2: *HTTP Status Code Semantics and Error Mapping*

CHAPTER 11: TESTING STRATEGY, TEST CASES, AND VERIFIED RESULTS

11.1 Testing Methodology and Multi-Layer Test Architecture

Software reliability and safety in healthcare systems demand rigorous verification across every application layer. MediGuide Care adopts a multi-tiered automated testing pyramid incorporating unit tests, JPA repository data-tier tests, Spring MVC controller integration tests with simulated HTTP requests, multi-threaded concurrency stress tests, and end-to-end user workflows. Automated tests run against an in-memory **H2 Database** configured in PostgreSQL compatibility mode (jdbc:h2:mem:mediguide_test;MODE=PostgreSQL) defined in backend/src/test/resources/application.properties. This eliminates dependencies on live network services or local PostgreSQL installations during test execution. Outbound email dispatch is safely mocked via TestMailConfig, preventing real SMTP network traffic while deterministically validating token generation and delivery triggers.

11.2 Unit, Service, Repository, and Controller Integration Tests

Every service class is covered by isolated unit tests utilizing Mockito mocks to verify business invariants, edge cases, and role restrictions. Controller integration tests utilize MockMvc to simulate HTTP requests with JSON payloads, validating request serialization, Jakarta Bean validation errors (HTTP 400), authentication filters (HTTP 401), role-based method security (HTTP 403), and proper HTTP response header generation (such as Retry-After on rate-limited resend requests).

11.3 Concurrency & Pessimistic Lock Verification

A dedicated multi-threaded test class, AppointmentConcurrencyTest, validates the double-booking prevention engine. Using Java's ExecutorService and CountDownLatch, multiple concurrent threads simulate simultaneous booking requests by different patients for the identical doctor availability interval. The tests verify that exactly one thread succeeds (returning HTTP 201 Created), while competing concurrent threads are serialized by the database pessimistic row-level lock and cleanly rejected with HTTP 409 Conflict. Zero duplicate appointments and zero thread deadlocks were observed across all test iterations.

11.4 Comprehensive Test Suite Execution Summary

The automated test suite was executed via Apache Maven (mvn test). All **298 automated tests passed successfully with 0 failures, 0 errors, and 0 skipped tests (100% pass rate)** in 2 minutes and 51 seconds. Table 11.1 presents a comprehensive matrix of 26 core representative test cases across all modules.

Test ID	Module / Area	Test Scenario / Input	Expected Result	Status
TC-01	Auth	Register user with valid patient payload	201 Created, BCrypt password, PAT- ID assigned	PASSED
TC-02	Auth	Register user with duplicate email	400 Bad Request, duplicate email rejected	PASSED
TC-03	Auth	Register with invalid role (ADMIN)	400 Bad Request, role restricted to PATIENT/DOCTOR	PASSED
TC-04	Auth	Login with valid credentials	200 OK, valid HMAC-SHA256 JWT returned	PASSED
TC-05	Auth	Login with incorrect password	401 Unauthorized, authentication fails safely	PASSED
TC-06	Security	Invoke protected API without JWT	401 Unauthorized, CustomAuthenticationEntryPoint	PASSED
TC-07	Security	Patient invoking doctor availability API	403 Forbidden, CustomAccessDeniedHandler	PASSED
TC-08	Email	Generate verification token on registration	Token created, 64-char SHA-256 hash saved in DB	PASSED
TC-09	Email	Verify email with valid raw token	200 OK, email_verified=true, token deleted	PASSED
TC-10	Email	Verify email with expired or invalid token	400 Bad Request, invalid/expired error message	PASSED
TC-11	Email	Resend verification within 60s cooldown	429 Too Many Requests, Retry-After header present	PASSED
TC-12	Email	Resend verification for non-existent email	200 OK uniform message (Anti-enumeration defense)	PASSED
TC-13	Doctor Search	Search doctors by medical specialty	200 OK, returns matching doctors case-insensitive	PASSED
TC-14	Doctor Search	Search doctors by city, area, PIN code	200 OK, combined criteria matched with AND logic	PASSED

Test ID	Module / Area	Test Scenario / Input	Expected Result	Status
TC-15	Privacy	Public doctor lookup by public ID	200 OK, clinic phone shown; personal phone excluded	PASSED
TC-16	Privacy	Patient updating own private phone number	200 OK, updated; excluded from public search	PASSED
TC-17	Availability	Doctor creating slot with start >= end	400 Bad Request, validation error thrown	PASSED
TC-18	Availability	Doctor creating overlapping slot	400 Bad Request, overlapping slot rejected	PASSED
TC-19	Availability	Unverified doctor creating slot	400 Bad Request, email verification required	PASSED
TC-20	Booking	Unverified patient booking appointment	400 Bad Request, email verification required	PASSED
TC-21	Booking	Patient booking valid available slot	201 Created, status=BOOKED, slot verified	PASSED
TC-22	Concurrency	Simultaneous concurrent booking on slot	Thread 1: 201 Created; Thread 2: 409 Conflict	PASSED
TC-23	Consultation	Doctor submitting consultation & Rx	201 Created, status transitions BOOKED->COMPLETED	PASSED
TC-24	History	Patient viewing own care timeline	200 OK, returns chronological past consultations	PASSED
TC-25	AI Summary	Doctor requesting AI summary without ack	400 Bad Request, clinical disclosure required	PASSED
TC-26	AI Summary	Doctor requesting AI summary with ack	200 OK, returns sanitized draft with disclaimer	PASSED

Table 11.1: *Core Automated Test Scenarios and Verified Execution Results*

11.5 Frontend Static Code Linting & Production Build Verification

The frontend client was subjected to static analysis and production compilation:

- **Static Code Linting:** Executed npm run lint via **Oxlint** across all 17 JavaScript and JSX source files using 104 rules. The linter finished in 182 ms, reporting **0 errors and 0 warnings**.
- **Production Build Verification:** Executed npm run build using **Vite 8.3.4**. All 34 client modules transformed and rendered successfully in 706 ms, producing an

optimized distribution bundle: dist/index.html (1.19 kB), dist/assets/index.css (67.69 kB / gzip: 11.14 kB), and dist/assets/index.js (341.21 kB / gzip: 93.97 kB).

CHAPTER 12: USER INTERFACE AND RESPONSIVE DESIGN

12.1 Design Philosophy, Color Palette, and Medical Theme

The MediGuide Care user interface is designed to evoke clinical professionalism, calm trust, and ergonomic clarity. The color scheme employs a carefully tailored palette: Deep Slate (#0F172A) provides a clean dark backdrop; Clinical Blue (#2563EB) establishes visual hierarchy for primary actions; Mint Green (#10B981) indicates verified status and active availability; Amber (#F59E0B) signals warnings and unverified email banners; and Crimson (#EF4444) highlights conflicts and errors. Typography utilizes Google Fonts *Outfit* for clean modern titles and *Inter* for highly legible body copy.

12.2 Modern Vanilla CSS Design System and Token Architecture

To maximize styling flexibility and avoid external utility bloat, the UI is built entirely upon a custom Vanilla CSS design system defined in index.css and RolePortal.css. CSS Custom Properties (variables) define standard tokens for typography, spacings, border radii, transitions, and glassmorphism backdrops (backdrop-filter: blur(12px)). Component styles are modularized, reusable, and predictable.

12.3 Doctor Portal Workflow and Slide-Over Clinical Drawer

The DoctorPortal component organizes physician workflows into distinct tabs: Profile Management, Availability Calendar, and Consultation Queue. The DoctorAvailabilityCalendar allows doctors to visually inspect schedules across monthly and daily grids. When consulting, the DoctorClinicalDrawer slides smoothly into view from the right margin, providing dedicated input fields for clinical notes, symptoms, and dynamic prescription line items. An integrated AI summary accordion allows doctors to view and copy historical clinical drafts without navigating away from the consultation form.

12.4 Patient Portal Workflow and Care Timeline

In PatientPortal, patients browse doctors using multi-field filter cards. Results clearly show the doctor's name, public ID, specialty, clinic name, physical address, and clinic telephone number for inquiries. Patients view active time slots and book appointments with real-time confirmation. The Care Timeline presents past consultations in chronological sequence, allowing patients to inspect diagnoses, instructions, and prescribed medications at any time. A dedicated Contact Profile tab permits patients to securely manage their private phone number.

12.5 Accessibility, Micro-Animations, and Viewport Adaptability

Interactive UI elements feature subtle micro-animations (0.15s cubic-bezier transitions on button hovers and drawer slide-overs) that provide immediate tactile feedback. Form fields include descriptive ARIA labels, semantic HTML5 tags, and visible focus rings. The layout utilizes CSS Flexbox and CSS Grid with fluid fractional units (minmax()), ensuring optimal usability across desktop monitors and modern tablet viewports (iPad / Surface).

CHAPTER 13: LIMITATIONS AND FUTURE ENHANCEMENTS

13.1 Operational Constraints and Current Limitations

In compliance with academic honesty, the following verified operational constraints of the current release are acknowledged:

1. **Synchronous Email Dispatch:** Email verification dispatch operates synchronously within registration and resend transactions. Although protected by graceful try-catch blocks (so SMTP delivery failures log a warning without aborting user registration), high-volume production deployments would benefit from asynchronous background queues.

2. **External SMTP Provider Quotas:** Real-world email delivery depends on third-party SMTP providers (such as Gmail, SendGrid, or Amazon SES). For example, free Gmail SMTP enforces standard daily sending limits (typically 500 emails/day) and requires a 16-character Google App Password with 2-Step Verification.

3. **Third-Party AI Dependency:** Generating live clinical history summaries requires an active internet connection and valid API credentials for an OpenAI-compatible provider. In local test environments, deterministic mocks and fallbacks are utilized.

4. **Database Schema Migration Tooling:** The current development configuration utilizes Hibernate's ddl-auto=update. In enterprise production environments, version-controlled migration frameworks (Flyway or Liquibase) should be adopted.

5. **Mobile Screen Polish:** While the responsive CSS grid performs smoothly on desktop and tablet viewports, specialized touch-first optimizations for compact smartphone screens are scheduled for the subsequent mobile sprint.

13.2 Technical Enhancements and Scalability Roadmap

Planned architectural advancements for subsequent iterations include:

- **Asynchronous Message Brokers:** Integrating RabbitMQ or Apache Kafka with dead-letter retry queues to offload email notifications, audit logging, and external AI calls from the main HTTP transaction thread.
- **Telemedicine Consultation Integration:** Implementing peer-to-peer WebRTC video and audio channels within the consultation drawer, enabling remote clinical examinations directly through the browser.
- **Automated SMS & WhatsApp Reminders:** Connecting an enterprise telephony gateway (Twilio / AWS SNS) to deliver automated SMS appointment reminders 24

hours prior to scheduled consultation times.

• **Pharmacy Fulfillment Module:** Providing a dedicated pharmacist portal where patients can share digital prescription tokens for direct medication dispensing and inventory tracking.

• **Containerization & Cloud Deployment:** Authoring Dockerfiles and Docker Compose manifests for containerized deployment across AWS ECS or Kubernetes with automated CI/CD pipelines.

CHAPTER 14: CONCLUSION

14.1 Summary of Academic Project Achievements

The MediGuide Care Doctor–Patient Consultation and Follow-up Management System successfully achieves all functional and architectural objectives established for this MCA mini-project. The project demonstrates the successful design and implementation of a modern, multi-tier web platform that transitions outpatient medical practices away from fragmented paper registers and ambiguous handwritten prescriptions. Key engineering milestones accomplished include:

- Successfully decoupled client presentation (React 19, Vite 8) from enterprise backend business logic (Spring Boot 3.2.5, Java 17).
- Implemented robust stateless security using HMAC-SHA256 JWT tokens, salted BCrypt password hashing, and fine-grained method-level RBAC.
- Developed a concurrency coordination engine utilizing database-level pessimistic write locking on ascending user IDs to guarantee zero appointment double-booking under concurrent traffic.
- Established a three-tier contact privacy architecture that protects patient phone numbers and doctor personal numbers while exposing public clinic contacts.
- Deployed an AI-assisted clinical history summarizer integrated with strict medical safety system prompts, character budgets, and doctor-in-the-loop review disclaimers.
- Engineered a cryptographic email verification subsystem with SHA-256 token hashing, 60-second cooldown rate limiting, and anti-enumeration protections.
- Validated the entire platform through an exhaustive automated test suite comprising **298 passing automated tests (100% pass rate)** and zero static lint errors.

14.2 Learning Outcomes and Viva Preparation Takeaways

Through the development of MediGuide Care, comprehensive practical competencies were gained across full-stack software engineering:

1. **Defensive Concurrency Control:** Gained deep mastery of relational transaction isolation, deadlock prevention algorithms, and row-level locking (SELECT ... FOR UPDATE) under concurrent web requests.
2. **Enterprise Security Engineering:** Learned how to implement stateless token-based authorization filters, manage token expiration and cryptographic signatures, and enforce progressive access policies.

3. **Domain Privacy Architecture:** Understood how to prevent sequential database enumeration through public alphanumeric IDs and how to segregate private personal data from public operational clinic contacts.

4. **Responsible Healthcare AI:** Developed practical techniques for sanitizing medical data payloads and engineering strict medical safety prompts that present AI outputs strictly as reviewable clinical drafts.

5. **Automated Testing Disciplines:** Mastered unit, repository, controller, and concurrency testing in Spring Boot using MockMvc, Mockito, and in-memory H2 PostgreSQL mode.

REFERENCES

[1] Pivotal Software & VMware Tanzu, “Spring Boot Reference Documentation (Version 3.2.x),” VMware, Inc., 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/

[2] Spring Security Team, “Spring Security Architecture and Reference Documentation (Version 6.2),” VMware Tanzu, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/

[3] PostgreSQL Global Development Group, “PostgreSQL 16.0 Documentation: Concurrency Control and Explicit Locking,” PostgreSQL.org, 2023. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[4] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force (IETF) RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[5] P. Resnick, Ed., “Internet Message Format,” Internet Engineering Task Force (IETF) RFC 5322, Oct. 2008. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc5322

[6] Meta Platforms, Inc., “React: A JavaScript Library for Building User Interfaces (Version 19),” React Documentation, 2024. [Online]. Available: https://react.dev/

[7] E. Evans, “Domain-Driven Design: Tackling Complexity in the Heart of Software,” Addison-Wesley Professional, Boston, MA, 2003.

[8] R. C. Martin, “Clean Architecture: A Craftsman's Guide to Software Structure and Design,” Prentice Hall, Upper Saddle River, NJ, 2017.

[9] National Institute of Standards and Technology (NIST), “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST Special Publication 800-63B, 2020.

[10] OpenAI, “OpenAI API Reference: Chat Completions and Structured Prompts,” OpenAI Developer Platform, 2024. [Online]. Available: https://platform.openai.com/docs/api-reference

APPENDIX

Appendix A: Environment Configuration Reference

MediGuide Care reads runtime credentials from environment variables to prevent secret leakage in version control. Table A.1 lists all configurable backend environment variables.

Environment Variable	Required	Default Value	Description / Purpose
DB_URL	No	jdbc:postgresql://localhost:5432/mediguide_db	JDBC connection URL for PostgreSQL
DB_USERNAME	No	postgres	Database authentication username
DB_PASSWORD	Yes	(Empty)	Database authentication user password
JWT_SECRET	Yes	(Empty)	HMAC-SHA256 signing secret key (min 32 chars)
JWT_EXPIRATION_MINUTES	No	15	Access token lifespan before expiration
SMTP_HOST	No	localhost	Outgoing SMTP mail server hostname
SMTP_PORT	No	587	SMTP port (587 STARTTLS, 465 SSL)
SMTP_USERNAME	Optional	(Empty)	SMTP username (e.g. Gmail address)
SMTP_PASSWORD	Optional	(Empty)	SMTP password or 16-char App Password
SMTP_FROM	No	no-reply@mediguide.local	Sender email address on verification emails
FRONTEND_URL	No	http://localhost:5173	Frontend client base URL for verification links
EMAIL_VERIFICATION_COOLDOWN_SECONDS	No	60	Rate limit cooldown between resend requests
EMAIL_VERIFICATION_EXPIRY_HOURS	No	24	Verification token validity duration
AI_API_KEY	Optional	(Empty)	API key for OpenAI-compatible LLM provider

Environment Variable	Required	Default Value	Description / Purpose
AI_BASE_URL	No	https://api.openai.com/v1	LLM API endpoint base URL
AI_MODEL	No	gpt-4o-mini	LLM model identifier for clinical summaries

Table A.1: *Backend Environment Variables Reference*

Appendix B: Sample API Request and Response Payloads

1. User Registration Request and Response:

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "fullName": "Jane Doe",
  "email": "jane.doe@example.com",
  "password": "SecurePassword123!",
  "role": "PATIENT"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqYW5lLmRvZUBleGFtcGxlLmNvb
SIs...",
  "role": "PATIENT",
  "email": "jane.doe@example.com",
  "fullName": "Jane Doe",
  "publicId": "PAT-9B2M1Q",
  "emailVerified": false
}
```

2. Appointment Booking Request:

```
POST /api/v1/appointments
Authorization: Bearer
Content-Type: application/json

{
  "availabilityId": 14,
  "startTime": "2026-11-01T10:00:00",
  "endTime": "2026-11-01T10:30:00"
}
```

Appendix C: Standard Error Response Schema

```
HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "timestamp": "2026-10-25T18:49:18.570",
  "status": 409,
  "error": "Conflict",
  "message": "The selected doctor already has a booked appointment ove
rlapping the requested interval [2026-11-01T10:00 - 2026-11-01T10:30]"
}
```